

■ Official University Master Textbook

Chapter 2: Python 3.12 Deep-Dive & CPython Memory Architecture

Section 1: CPython Architecture & Execution Model

1.1 How CPython Executes Code

Python is an interpreted, dynamically-typed programming language implemented primarily in C (CPython). When you run 'python script.py', CPython performs 4 steps:

1. **Lexing & Parsing:** Converts source text into an Abstract Syntax Tree (AST).
2. **Bytecode Compilation:** Compiles AST into platform-independent bytecode instructions (.pyc cached files).
3. **Evaluation Loop (ceval.c):** Executes bytecode stack machine instructions.
4. **PyObject Memory Management:** Allocates and frees memory structures using Reference Counting and a Cyclic Garbage Collector.

1.2 PyObject Structure & Memory Allocator (PyMalloc)

In CPython, everything is an object. Every variable points to a C struct called PyObject containing two header fields:

- **ob_refcnt:** Reference count tracking how many pointers target this object.
- **ob_type:** Pointer to the type object defining methods and data size.

CPython uses PyMalloc, a small-object memory allocator that manages 256KB Arenas split into 4KB Pools to avoid OS malloc overhead.

Section 2: The Global Interpreter Lock (GIL) & Concurrency

2.1 What is the GIL?

The Global Interpreter Lock (GIL) is a mutual exclusion lock used by CPython to prevent multiple native CPU threads from executing Python bytecodes at the same time. The GIL guarantees thread safety for CPython's reference counting mechanism.

- **I/O-Bound Tasks (MQTT / Sockets / File Writes):** CPython releases the GIL during network I/O wait times, allowing multi-threaded speedups.
- **In This Project:** run_system.py uses threading.Thread(target=run_simulator_background, daemon=True) to run 1-second telemetry generation in parallel with Streamlit without blocking the UI.

Section 3: Core Standard Library Modules Used

3.1 Built-in Modules Breakdown

- **sqlite3:** Implements C-bindings for serverless SQL storage.
- **subprocess:** Executes external operating system binaries (pip install, streamlit run).
- **threading:** Manages OS-level background daemon threads.
- **json:** Performs fast serialization (json.dumps) and deserialization (json.loads) between dicts and UTF-8 strings.